SQL Server 2025 Vectors

Under the Hood and Into Production

Anthony Nocentino



Anthony E. Nocentino

Senior Principal Field Solution Architect, Everpure · Global Database Lead

What I work on

- System architecture and performance
- SQL Server, Kubernetes, and containers
- Microsoft Data Platform MVP
- Pluralsight author



nocentino.com · github.com/nocentino · anocentino@everpuredata.com

Agenda

1 Vectors 101: the math you need

2 Inside the VECTOR type

3 **Demo 1: vector internals**

4 Vector indexes: where they help and where they don't

5 **Demo 2: index performance at scale**

6 Into production: patterns and the app

7 **Demo 3: the RAG application**

8 Takeaways, then questions

Before we start

Level 300: you know SQL Server internals. You don't need any machine learning.

You already know

- Reading execution plans and `STATISTICS IO`
- Pages, B-tree indexes, the buffer pool
- Filegroups, the log, and backups
- Stored procedures and permissions

We'll build from here

- What an embedding is
- Distance: cosine, dot, Euclidean
- Exact vs approximate search, and recall
- DiskANN, at the depth a DBA needs

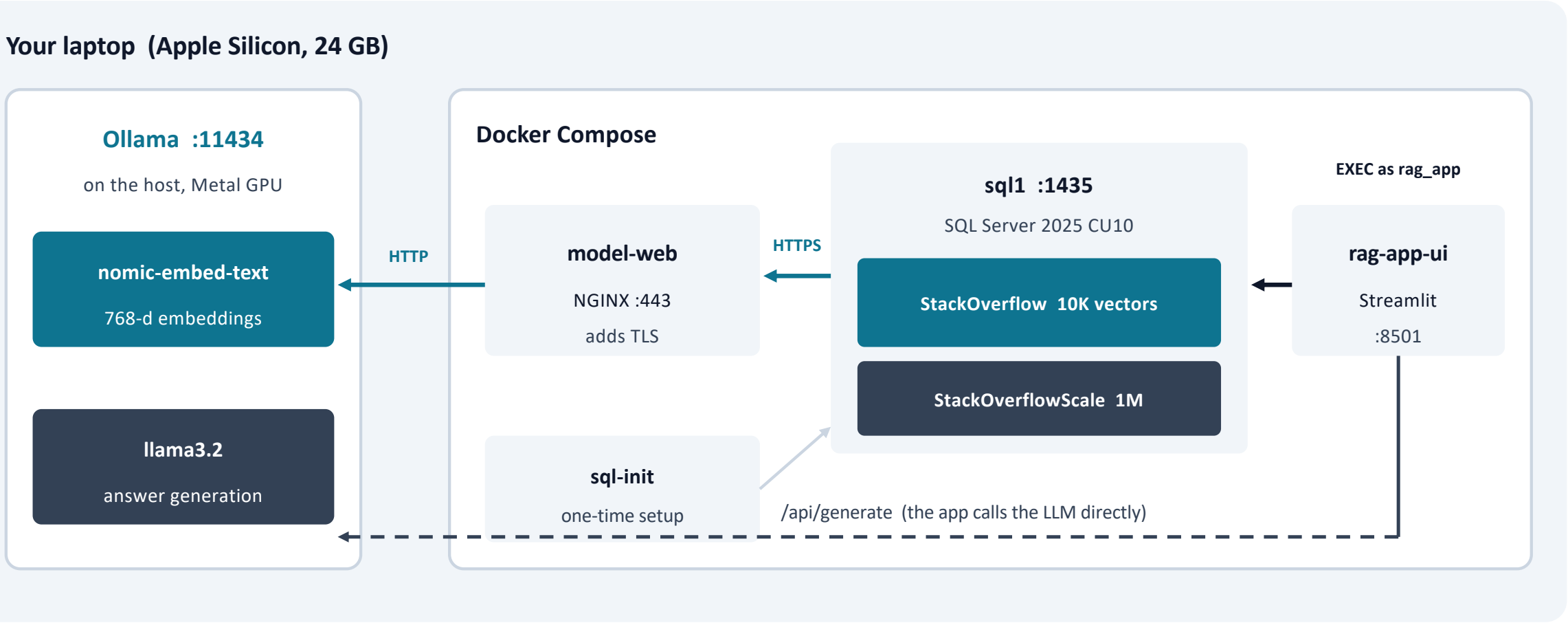
You don't need

- Machine learning or linear algebra
- A cloud account or API keys
- Python: the app is a tour, not a lesson

Everything runs on a laptop. Follow along: github.com/nocentino/vector-under-the-hood

The test lab

One laptop: SQL Server 2025 CU10 in Docker, Ollama on the host, no cloud



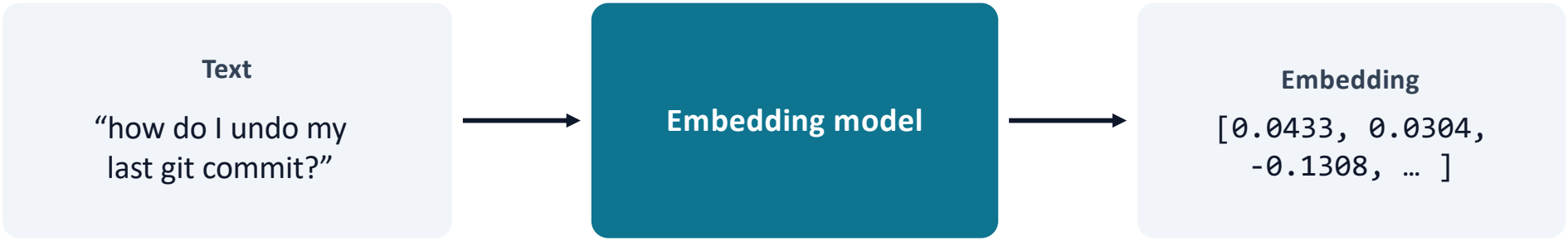
Vectors 101

The math you need for the next hour



From text to a vector

Meaning in, a fixed-length list of numbers out. Three ideas carry the rest of the session.



A vector is a list of numbers

An embedding model turns text into 768 numbers.

- Stored in a VECTOR(768) column
- The model picks the count, not you

Direction is meaning

Text with similar meaning points the same way.

- Our model's vectors all have length 1
- So only the angle between them matters

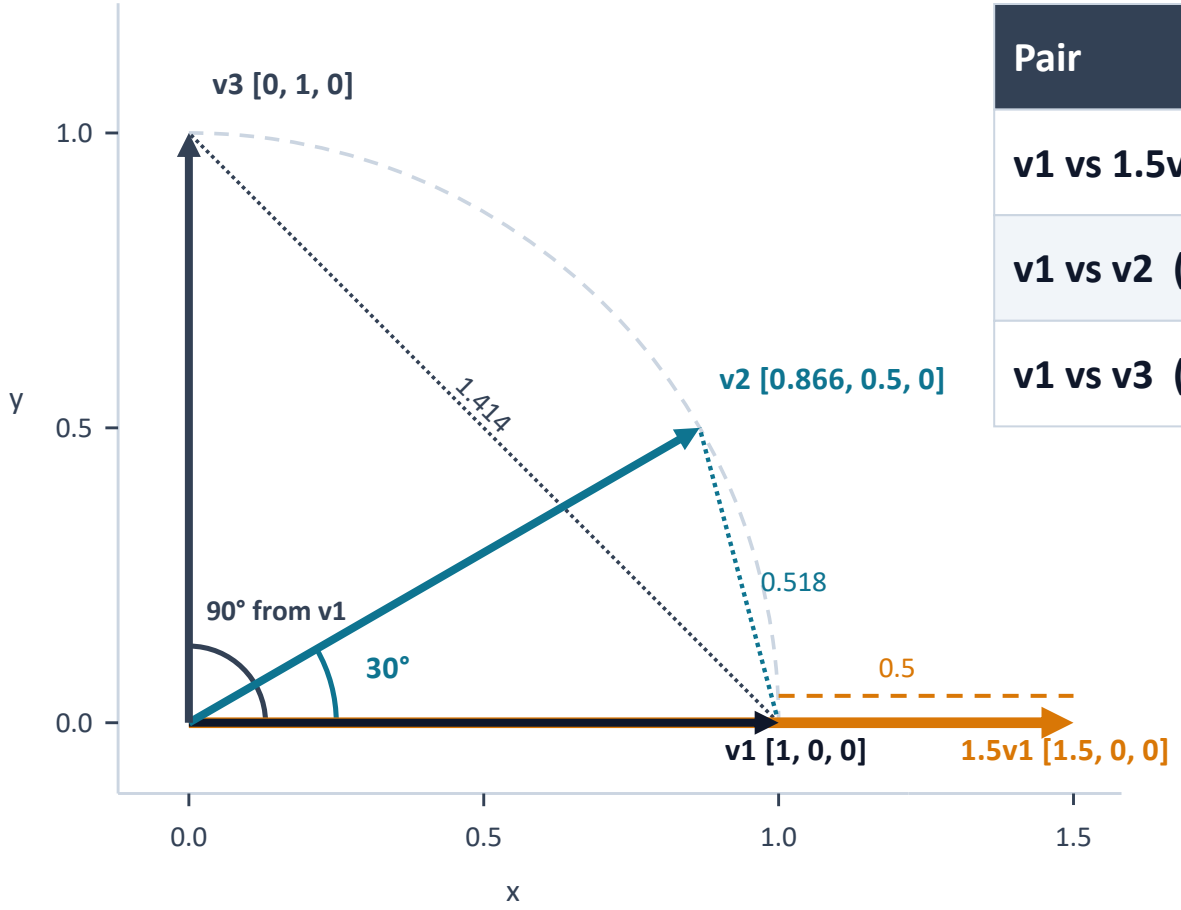
Search is a sort by distance

Embed the question, measure, keep the closest K.

- VECTOR_DISTANCE: lower is closer
- Strong match ~0.12
- Unrelated ~0.68

Three vectors, three distances

Lower is always closer. For text, use cosine; dot and Euclidean matter only when length means something.



Pair	cosine distance	euclidean	dot
v1 vs 1.5v1 (0°)	0	0.5	−1.5
v1 vs v2 (30°)	0.134	0.518	−0.866
v1 vs v3 (90°)	1	1.414	0

VECTOR_DISTANCE flips similarity so lower is closer:
cosine distance = 1 − cos θ

cos θ (similarity)	1 − cos θ (distance)
cos 0° = 1	**1 − 1 = 0**
cos 30° = 0.866	**1 − 0.866 = 0.134**
cos 90° = 0	**1 − 0 = 1**

Inside the VECTOR type

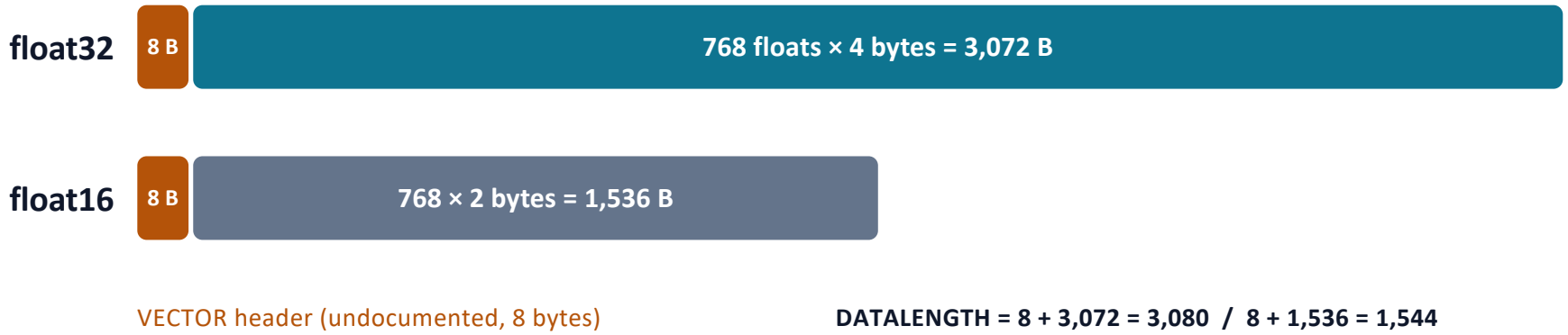
Storage layout, precision, and exact search



Anatomy of a VECTOR

IEEE 754 floats behind an 8-byte header, stored in the row

One VECTOR(768) value: an 8-byte header, then the floats



One element, bit by bit (IEEE 754, little-endian on disk)



Why a DBA cares about this data

Each ~3 KB vector doesn't compress, can't be sorted, and is read in full. At 1M rows they grew the database 3.7×.

Buffer pool

- An exact search reads every vector page
- 1M rows: ~535,000 pages (4.1 GB) per query

Scans

- Distance needs every row; no predicate skips one
- Cost grows with every row you add

Indexes

- A vector can't be a B-tree key, compared, or sorted
- Only a vector index (a graph) helps

Backups

- Floats look like noise to the compressor
- ZSTD 1.41:1 with vectors, 1.88:1 without

Compression

- ROW and PAGE compression save nothing
- 2,000 vectors: 1,021 pages, 1,024 with PAGE

The log

- ~3.6 KB of log per inserted row
- 1M rows: ~3.6 GB of log per load

Dimensions set the size

VECTOR(n) stores n numbers behind the same 8-byte header

Embedding model	Dimensions	VECTOR(n) float32	VECTOR(n, float16)
nomic-embed-text (our demos)	768	3,080 B	1,544 B
mxbai-embed-large	1,024	4,104 B	2,056 B
OpenAI text-embedding-3-small	1,536	6,152 B	3,080 B
OpenAI text-embedding-3-large	3,072	won't fit (Msg 2717)	6,152 B
The limit	1,998 / 3,996	8,000 B	8,000 B

The cap is 8,000 bytes, not a dimension count: 1,998 four-byte numbers or 3,996 two-byte ones. That's why float16 fits twice the dimensions.

Measured with DATALENGTH on SQL Server 2025 CU10. Size grows in a straight line with dimensions.

float32 or float16?

Use float16 when storage or dimensions are the limit and your vectors are unit length. Otherwise float32.

float32 (the default)

4 bytes per element, up to 1,998 dimensions.

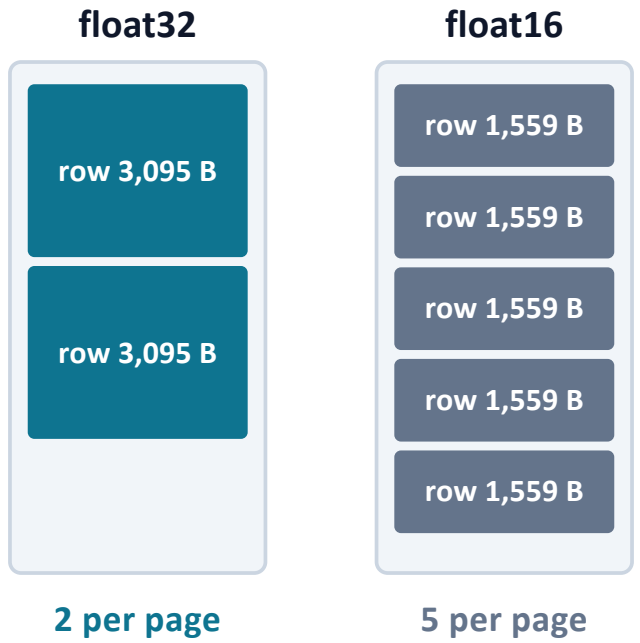
- Generally available in SQL Server 2025
- Full precision for any workload
- Every client driver handles it

float16 (preview)

2 bytes per element, up to 3,996 dimensions.

- Half the storage and I/O; distances moved ~0.00001
- Fits 3,072-d models
- Needs PREVIEW_FEATURES; binary transport needs ODBC 18.7.1+ or JDBC 13.4+
- Can't mix with float32 in VECTOR_DISTANCE

768 dims on one 8 KB page



Same page, more vectors: 1,000 vectors at 768 dims in a test table with only an INT key and the vector took 500 pages as float32 and 200 as float16. An exact scan reads 60% fewer pages.

How SQL Server runs a similarity search

Exact kNN (VECTOR_DISTANCE)

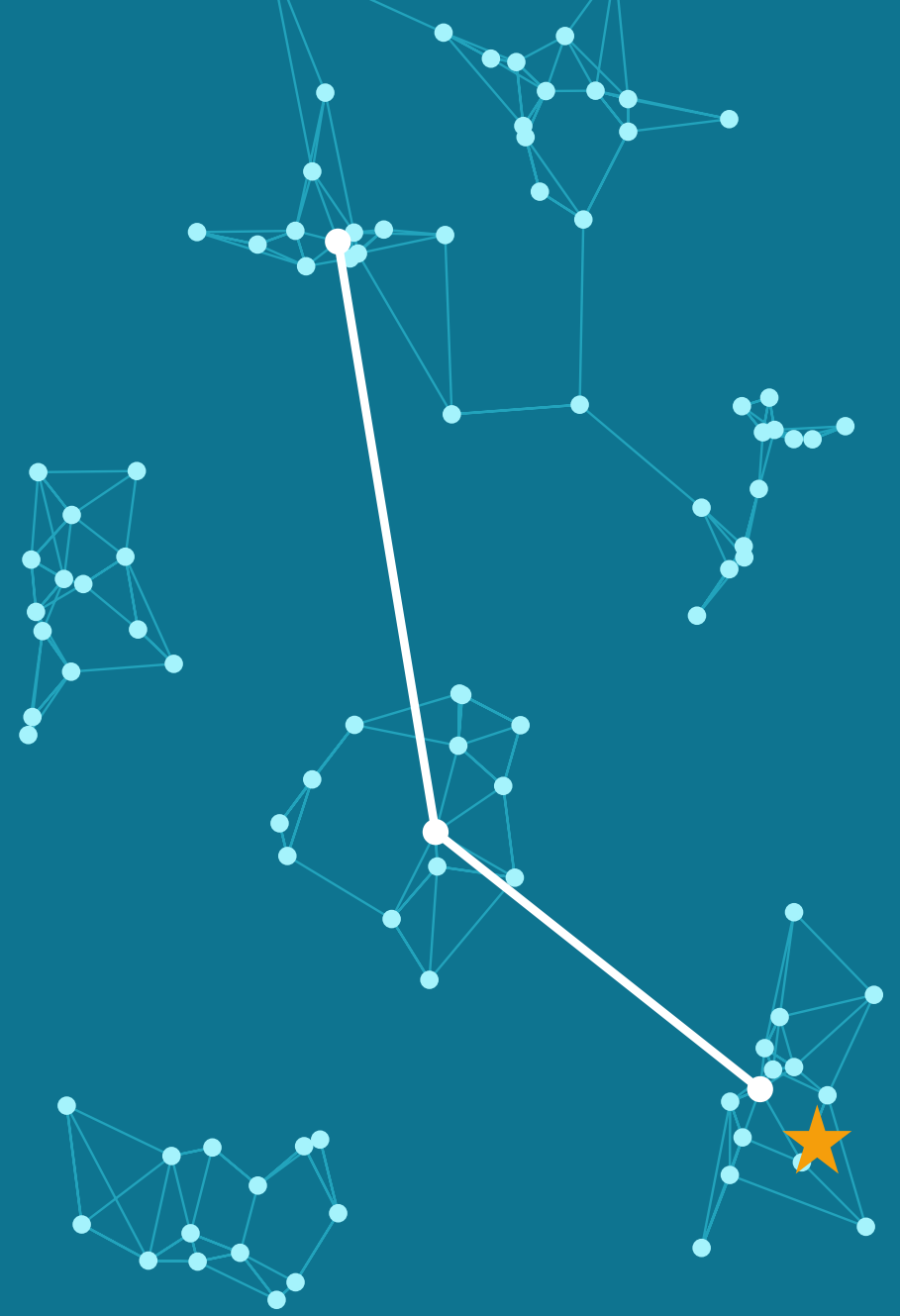


Data flows left to right here (SSMS draws it right to left). Every row is read and every distance computed before the sort keeps 10.

LIVE DEMO

Demo 1: Vector internals

Files and pages · decode a vector off the page · float16 · distance · the exact-search plan



Vector indexes

Where they help, and where they don't



Exact kNN vs approximate ANN

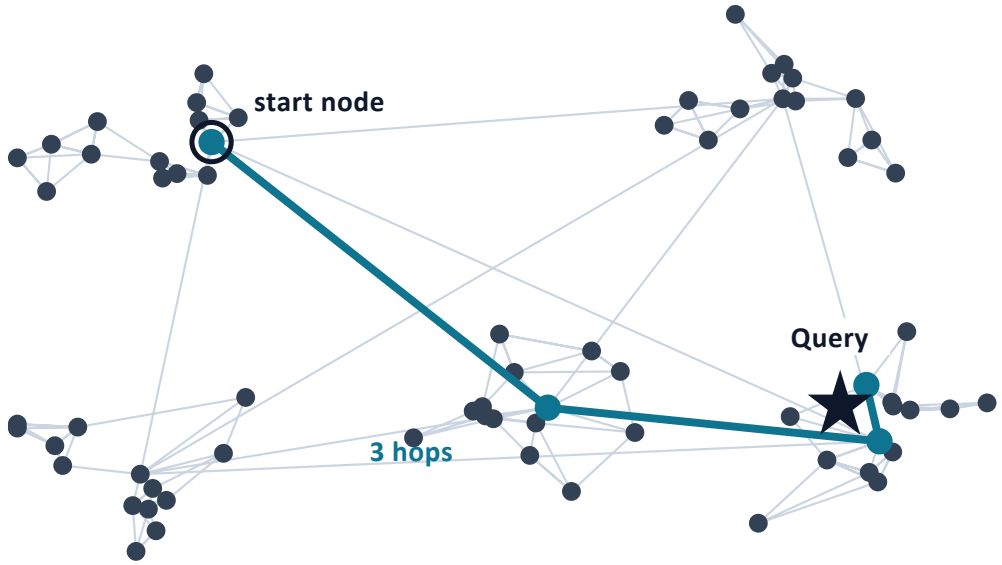
Exact visits every vector. ANN walks a graph and visits a few.

Exact kNN: measure every vector



Every row, every time. O(n). Perfect recall.

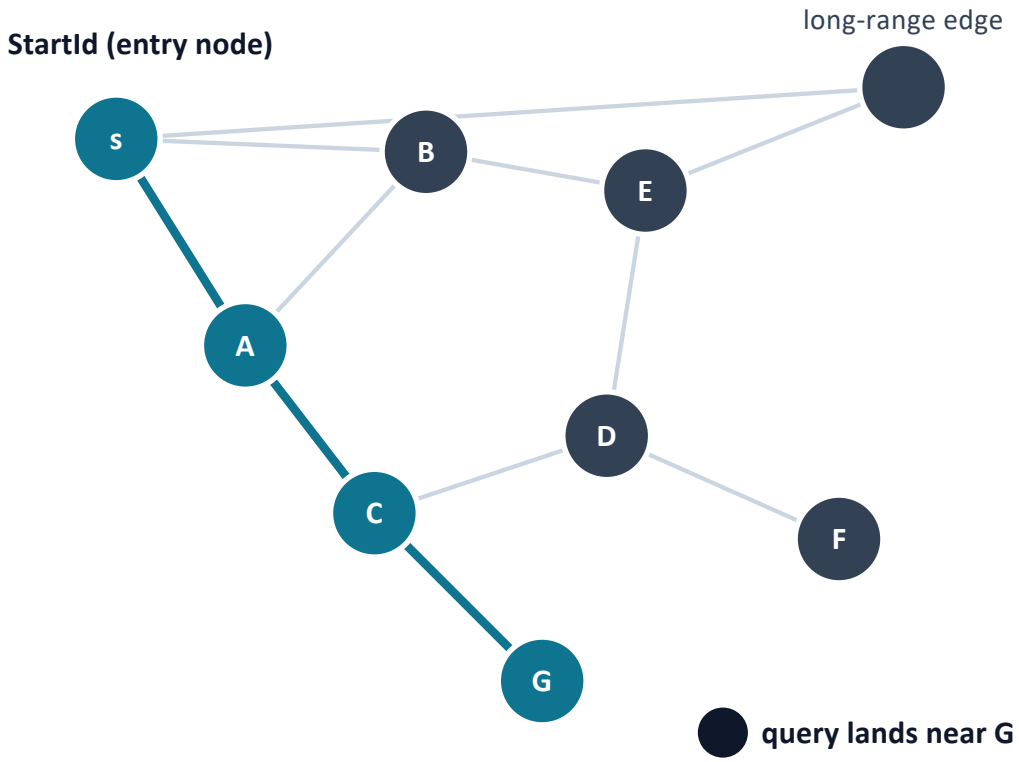
ANN (DiskANN): walk a graph



A few hops toward the query. Far fewer reads. Approximate.

Inside DiskANN

A graph built once over every vector, walked greedily for each query



Greedy search

Start at the entry node, hop to the neighbor closest to the query, stop when nothing is closer.

R = 48, L = 48

At most 48 edges per node; a 48-entry candidate list during the build. Read from sys.vector_indexes.

Stored beside the table

Graph edges live in an internal table: about 3 MB for 10K rows, 261 MB for 1M. The quantization table is empty on SQL Server 2025.

Built, not maintained

CREATE VECTOR INDEX builds the whole graph. 10K rows: about 17 s at MAXDOP 4. 1M rows: 35 min.

Two plans for the same question

The index replaces the scan, the per-row distance, and the sort with one seek.

Exact kNN (VECTOR_DISTANCE)



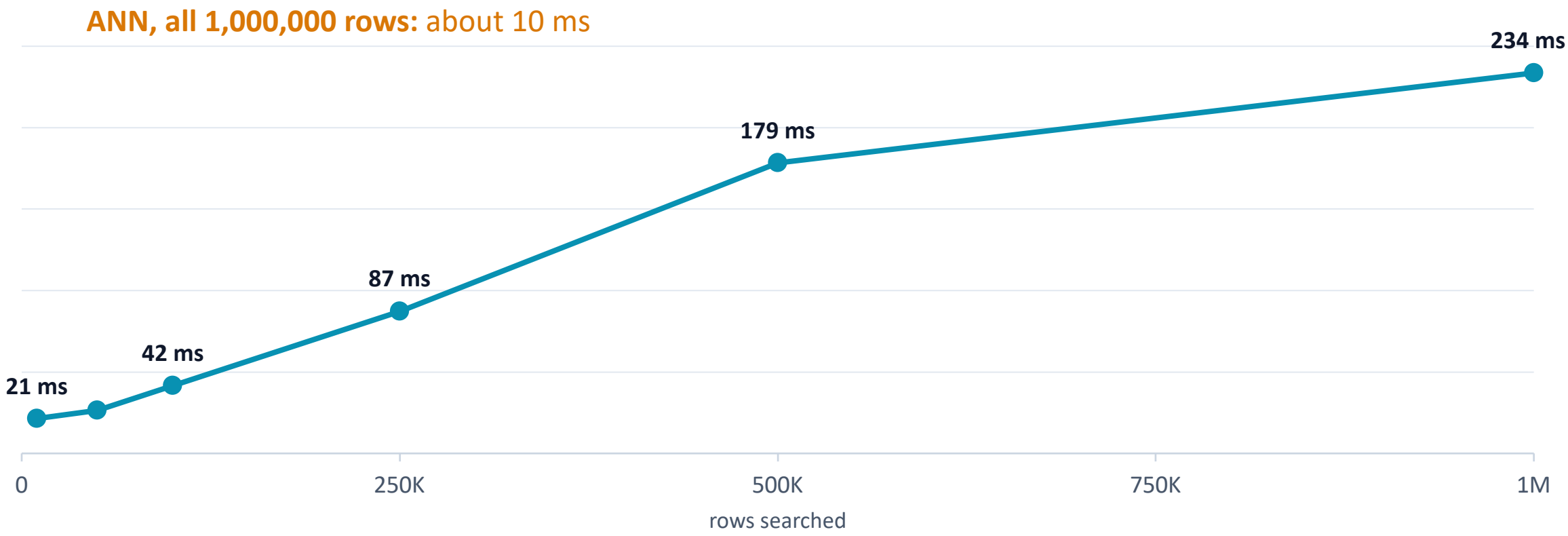
Approximate ANN (VECTOR_SEARCH)



Data flows left to right. The ANN plan runs serial and hands rows back in distance order, so nothing sorts. With WHERE p.Score > 5000 and TOP_N = 2,500, the seek streamed 1,889 candidates before 5 passed.

Exact search grows with every row

One exact kNN query on this laptop: average elapsed time by rows searched. Watch the shape, not the ms.



Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

Recall: the price of approximate

How many of the true top 10 the index found, at 1,000,000 rows

Search	Recall@10	Time per query	Logical reads (avg)
ANN, TOP_N = 10	0.84 – 0.89	8 – 15 ms	~6,400
ANN, TOP_N = 50, keep 10	0.96 – 0.98	24 – 28 ms	~20,800
Exact kNN	1.0	175 – 265 ms	535,731

One question, live in Demo 2: “How do I make an HTTP request in javascript” found 9 of the true top 10 with TOP_N = 10, and all 10 with TOP_N = 50. Recall falls as the table grows (0.96 at 10,000 rows). Ask for more candidates and keep the closest: that's oversampling, and the demo app does it 5×.

Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

What an index build costs you

Building the index over 100,000 vectors on 12 cores. MAXDOP trades build time for room on the box.

Build	Elapsed	CPU	Memory grant	Exact kNN during the build
Default (all 12 cores)	140 s	~1,520 s	121 MB	~800 ms
WITH (MAXDOP = 4)	237 s	914 s	63 MB	~250 ms
WITH (MAXDOP = 1)	908 s	885 s	31 MB	~110 ms

Reads keep working

- SELECT and exact kNN run, starved for CPU
- Baseline kNN: about 40 ms

Writes and ANN don't

- INSERT, UPDATE fail with Msg 42231 at once
- VECTOR_SEARCH: Msg 42227 until done

Small footprint

- Used ~26 MB of memory at any MAXDOP
- tempdb under 10 MB; graph 26 MB

SQL Server 2025 CU10, Developer and Standard: same behavior. Only the index MAXDOP option was honored; the known-issues page says the reverse, and Demo 2 optional Appendix A has the test.

Limits on SQL Server 2025, and what Azure SQL changes

The index is preview on 2025 and GA on Azure SQL, where the latest version removes all three limits.

	SQL Server 2025: earlier index, preview	Azure SQL: latest index, GA
Writes	**Read-only table** INSERT, UPDATE, DELETE fail with Msg 42231. Loading = drop the index, insert, rebuild. Demo 3 runs that cycle.	**Full DML** Changes join the graph in the background. sys.dm_db_vector_indexes shows how far behind it is.
Filters	**Walk, then filter** `VECTOR_SEARCH(…, TOP_N = 5) … WHERE p.Score > 5000` The walk returns 5, then WHERE runs: 0 rows, no error. You guess TOP_N: 2,500 gets 5 back.	**Filter during the walk** `SELECT TOP (5) WITH APPROXIMATE … WHERE p.Score > 5000` Keeps walking until 5 rows qualify. No TOP_N to guess.
Syntax	**TOP_N, deprecated** This earlier index version will be retired.	**TOP (N) WITH APPROXIMATE** TOP_N fails with Msg 42274. The optimizer picks ANN or kNN; FORCE_ANN_ONLY overrides it.

Keep vector queries in procs, and moving to the new syntax happens in one place.

kNN or ANN: which, when

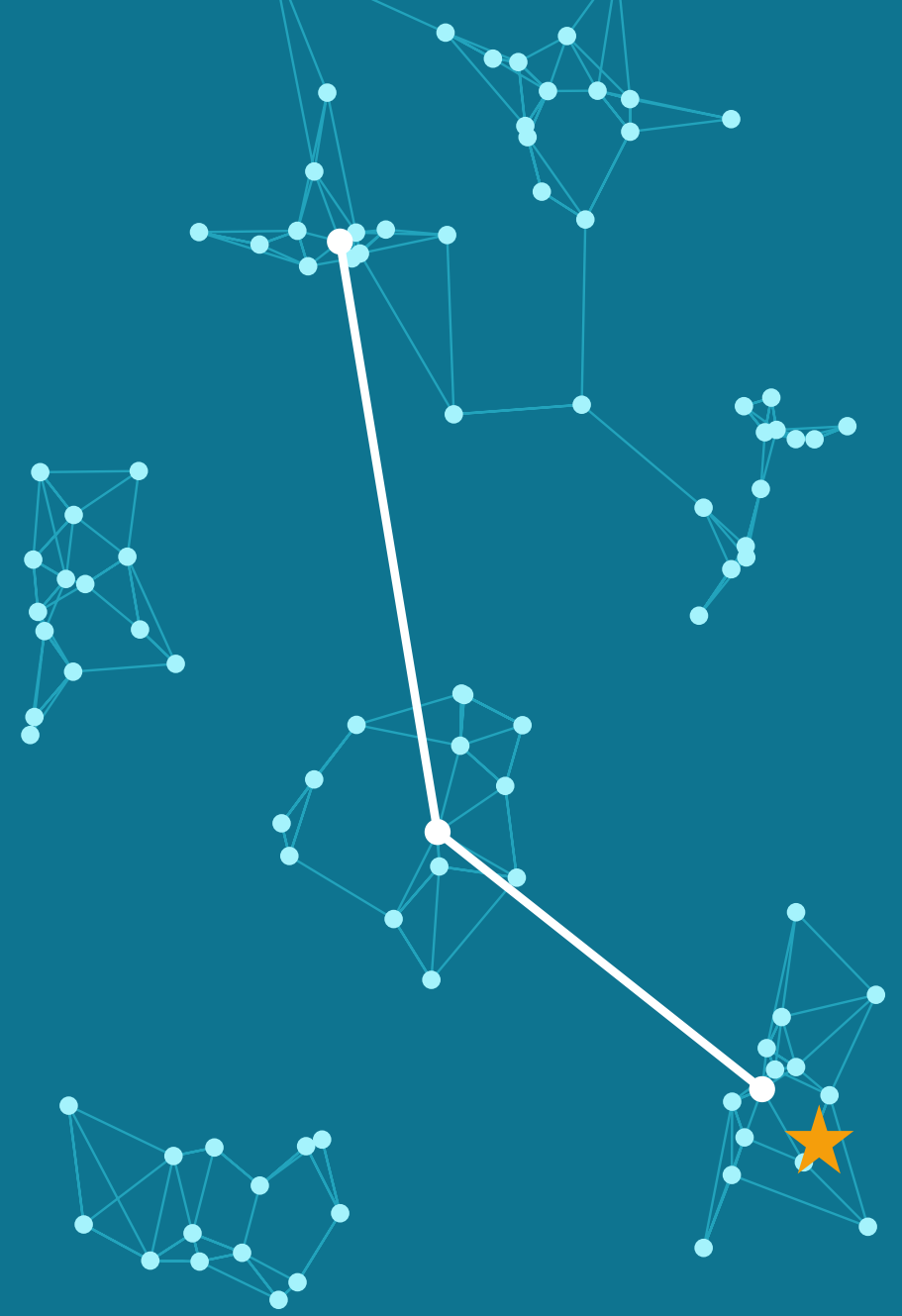
Microsoft's guidance: exact search is fine up to about 50,000 vectors

Situation	Use	Why
Under ~50,000 vectors, or a filter cuts it that far	Exact kNN	Fast enough, perfect recall, nothing to maintain
A small table (about 1,000 rows)	Exact kNN	The index costs more reads than the scan
Hundreds of thousands to millions of vectors	ANN	535,731 vs 5,564 reads for one question at 1M rows
You need every true neighbor	Exact kNN	ANN found 84–89% of the true top 10 at 1M rows
A selective filter, on SQL Server 2025	kNN, or ANN with TOP_N ≥ K ÷ pass rate	Filters run after the walk; a small TOP_N can return 0 rows
The table changes all day	kNN, or ANN on Azure SQL	The 2025 index makes the table read-only

LIVE DEMO

Demo 2: Performance at scale

Build and inspect the index · kNN vs ANN · 1,000,000 rows · recall · the limits



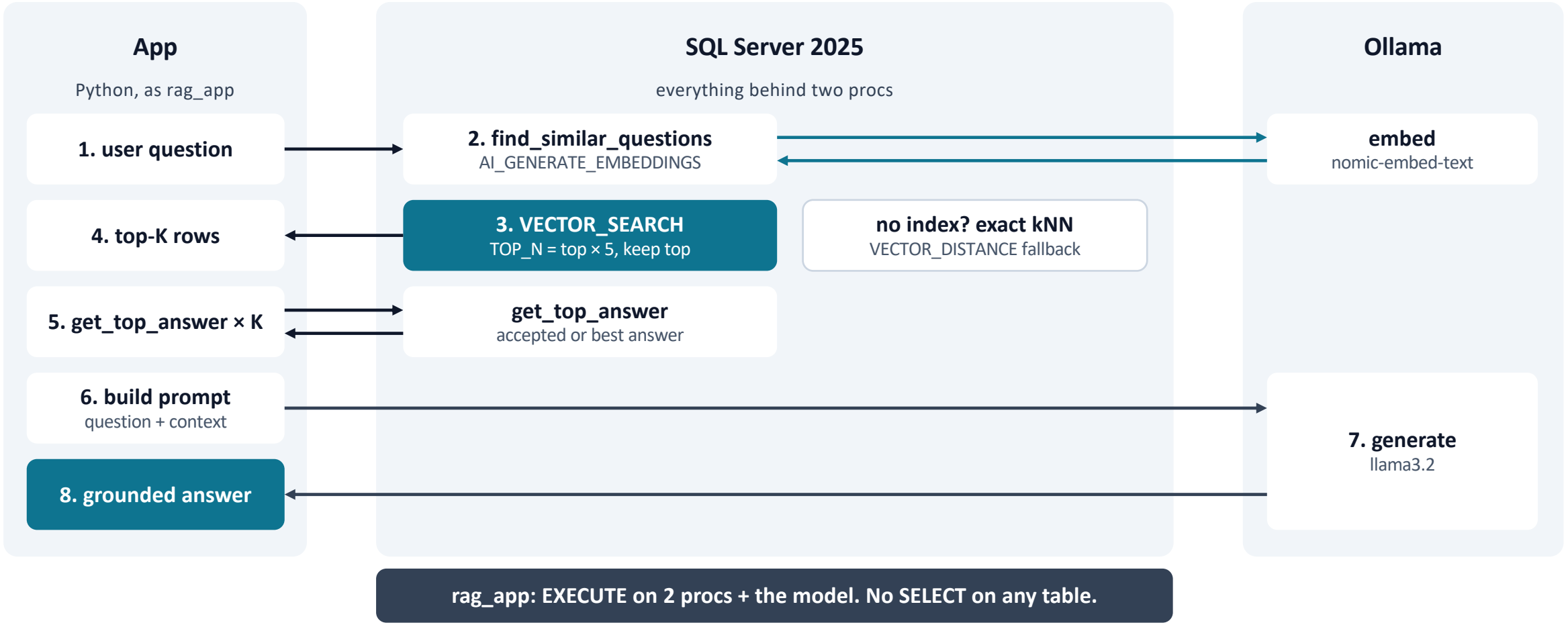
Into production

Build a working application on all of it



RAG application architecture

The app sends text and gets rows back. It never sees a vector.



Patterns that make it production-ready

What Demo 3 builds, and why

Satellite table

Vectors in their own table and filegroup.

- Single INT key (required on SQL Server 2025)
- Rebuild without touching Posts
- chunk = what was embedded

Procs as the contract

The app calls two procs.

- Exact fallback with no index
- Oversamples 5× for recall
- Swap models in one place

Least privilege

rag_app can EXECUTE, not SELECT.

- VIEW DEFINITION so the proc sees the index
- No access to the vectors
- Default database: StackOverflow

Operate it

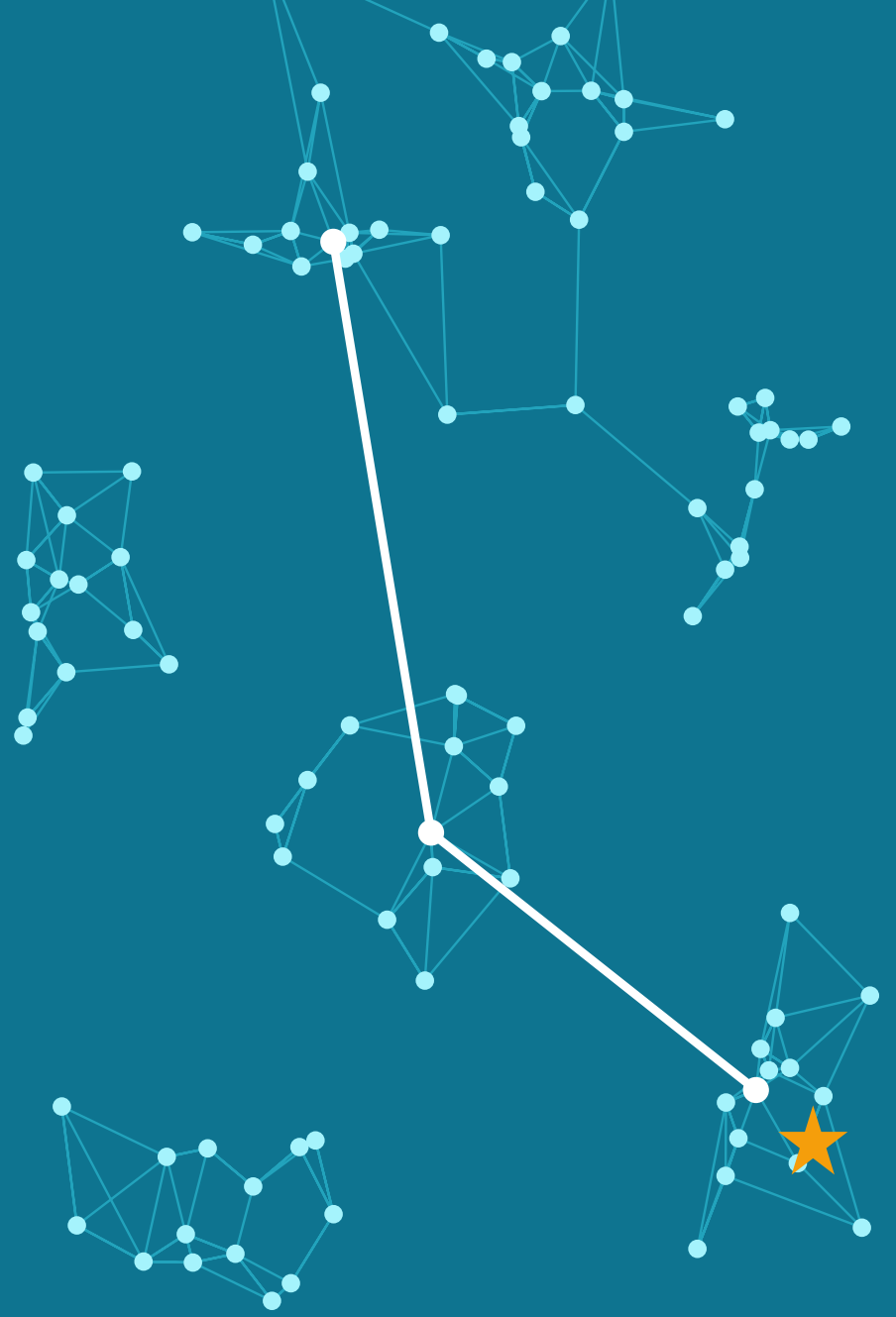
Drop, load, rebuild on a schedule.

- The fallback isn't free at 1M rows
- Check for stale chunks
- Alert when the app reports kNN

LIVE DEMO

Demo 3: The RAG application

Schema · the search proc · least privilege · drop, load, rebuild · the app



Takeaways

Measure on your data, your model, your hardware

Storage

A VECTOR is a native float array in the row.

- float16 halves it for ~0.00001 of distance
- Embedding costs ~15 ms a row*

Indexes

Know where the line is.

- Exact search under ~50,000 vectors
- ANN at 1M rows: 5,564 reads, not 535,731
- Oversample to keep recall
- Filters run after the walk: oversample or get 0 rows

Production

Put it behind procs.

- Least privilege by default
- Plan for read-only tables on 2025
- Watch the fallback path

** Environment specific: nomic-embed-text on Ollama, Apple M2 GPU, through a local NGINX TLS proxy. Your model, hardware, and network will differ, so measure it.*

Questions?

github.com/nocentino/vector-under-the-hood

Every demo, the lab, and the app, running on your laptop

- Vector data type: learn.microsoft.com/sql/t-sql/data-types/vector-data-type
- CREATE VECTOR INDEX: learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql
- VECTOR_SEARCH: learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql
- DiskANN: Subramanya et al., NeurIPS 2019

Your feedback is important to us



Evaluate this session at:

passdatacommunitysummit.com/evaluations